

INDIRA GANDHI DELHI TECHNICAL UNIVERSITY FOR WOMEN



Academic Year: 2024-2028

COMPUTER SCIENCE AND ENGINEERING

DATA STRUCTURES AND ALGORITHM

Practical File

(Subject Code: BCS101)

Submitted to :

Prof Name

Designation

COMPUTER SCIENCE AND ENGINEERING

Submitted by :

NAME

Enrollment No: 123456789

Course: B.TECH

Semester: 4

Index

E. No.	Aim	Date	Sign
1	Reverse a Linked List		
2	Delete the Last Node from a Linked List		
3	Insert a Node at the Beginning, Middle, and End of a Linked List		
4	Search an Element in a Linked List		
5	Implement Stack using Arrays		
6	Implement Queue using Linked Lists		
7	Convert an Infix Expression to Postfix Expression using Stack		
8	Evaluate a Postfix Expression using Stack		
9	Perform Breadth First Search (BFS) Traversal of a Graph		
10	Perform Depth First Search (DFS) Traversal of a Graph		
11	Implement Binary Search Tree (BST) and Perform Inorder, Preorder, and Postorder Traversals		
12	Implement Quick Sort and Compare its Time Complexity with Merge Sort		

Experiment 1

AIM

Reverse a Linked List

Algorithm

1. Define a Node structure to represent each element in the linked list.
2. Create a function to insert nodes at the end of the list.
3. Create a function to reverse the linked list by initializing three pointers: previous, current, and next.
4. Traverse the list, updating the next pointer of each node to point to the previous node.
5. Update the previous and current pointers in each iteration.
6. After the loop, update the head of the list to point to the new first node, which is the last node in the original list.

Source Code

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert a node at the end of the list
void insertNode(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
        return;
    }

    struct Node* lastNode = *head;
    while (lastNode->next) {
        lastNode = lastNode->next;
    }

    lastNode->next = newNode;
}

// Function to reverse the linked list
void reverseList(struct Node** head) {
    struct Node* prev = NULL;
    struct Node* current = *head;
    struct Node* next = NULL;
```

```

while (current) {
    next = current->next;
    current->next = prev;
    prev = current;
    current = next;
}

*head = prev;
}

// Function to print the linked list
void printList(struct Node* node) {
    while (node) {
        printf("%d -> ", node->data);
        node = node->next;
    }
    printf("NULL\n");
}

int main() {
    struct Node* head = NULL;

    insertNode(&head, 1);
    insertNode(&head, 2);
    insertNode(&head, 3);
    insertNode(&head, 4);
    insertNode(&head, 5);

    printf("Original List: ");
    printList(head);

    reverseList(&head);

    printf("Reversed List: ");
    printList(head);

    return 0;
}

```

Output

Original List: 1 -> 2 -> 3 -> 4 -> 5

Reversed List: 5 -> 4 -> 3 -> 2 -> 1

Experiment 2

AIM

Delete the Last Node from a Linked List

Algorithm

1. Define a structure for the linked list node.
2. Initialize the linked list with some values.
3. Check if the list is empty.
4. If not, traverse to the second last node.
5. Update the next pointer of the second last node to NULL.
6. Free the memory of the last node.

Source Code

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

void deleteLastNode(Node** head) {
    if (*head == NULL) {
        printf("List is empty\n");
        return;
    }
    if ((*head)->next == NULL) {
        free(*head);
        *head = NULL;
        return;
    }
    Node* temp = *head;
    while (temp->next->next != NULL) {
        temp = temp->next;
    }
    Node* lastNode = temp->next;
    temp->next = NULL;
    free(lastNode);
}

void printList(Node* head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    Node* head = (Node*)malloc(sizeof(Node));
    head->data = 1;
    Node* temp = head;
    for (int i = 2; i <= 5; i++) {
        Node* newNode = (Node*)malloc(sizeof(Node));
        newNode->data = i;
        temp->next = newNode;
        temp = temp->next;
    }
}
```

```
}  
printf("Linked List before deletion: ");  
printList(head);  
deleteLastNode(&head);  
printf("Linked List after deletion: ");  
printList(head);  
return 0;  
}
```

Output

Linked List before deletion: 1 2 3 4 5

Linked List after deletion: 1 2 3 4

Experiment 3

AIM

Insert a Node at the Beginning, Middle, and End of a Linked List

Algorithm

1. Create a Node class with an integer data field and a next field to point to the next node in the linked list.
2. Create an insertAtBeginning function that takes the head of the list and the data to be inserted as parameters.
3. Create an insertAtMiddle function that takes the head of the list, the position, and the data to be inserted as parameters.
4. Create an insertAtEnd function that takes the head of the list and the data to be inserted as parameters.
5. Create a printList function that takes the head of the list as a parameter and prints the data of all nodes in the list.

Source Code

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
typedef struct Node {
    int data;
    struct Node* next;
} Node;

// Function to create a new node
Node* createNode(int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    if (!newNode) {
        printf("Memory error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert a node at the beginning of the linked list
Node* insertAtBeginning(Node* head, int data) {
    Node* newNode = createNode(data);
    if (head == NULL) {
        return newNode;
    }
    newNode->next = head;
    return newNode;
}

// Function to insert a node at the middle of the linked list
Node* insertAtMiddle(Node* head, int pos, int data) {
    Node* newNode = createNode(data);
    if (head == NULL || pos == 0) {
        newNode->next = head;
        return newNode;
    }
    Node* current = head;
    for (int i = 0; i < pos - 1; i++) {
        if (current->next == NULL) {
            break;
        }
    }
```

```

        current = current->next;
    }
    newNode->next = current->next;
    current->next = newNode;
    return head;
}

// Function to insert a node at the end of the linked list
Node* insertAtEnd(Node* head, int data) {
    Node* newNode = createNode(data);
    if (head == NULL) {
        return newNode;
    }
    Node* current = head;
    while (current->next != NULL) {
        current = current->next;
    }
    current->next = newNode;
    return head;
}

// Function to print the linked list
void printList(Node* head) {
    while (head != NULL) {
        printf("%d ", head->data);
        head = head->next;
    }
    printf("\n");
}

int main() {
    // Create a sample linked list: 20 -> 30 -> 40
    Node* head = createNode(20);
    head->next = createNode(30);
    head->next->next = createNode(40);

    // Insert node with data 10 at the beginning
    head = insertAtBeginning(head, 10);
    printf("Linked list after insertion at beginning: ");
    printList(head);

    // Insert node with data 25 at the middle
    head = insertAtMiddle(head, 2, 25);
    printf("Linked list after insertion at middle: ");
    printList(head);

    // Insert node with data 50 at the end
    head = insertAtEnd(head, 50);
    printf("Linked list after insertion at end: ");
    printList(head);

    return 0;
}

```

Output

Linked list after insertion at beginning: 10 20 30 40

Linked list after insertion at middle: 10 20 25 30 40

Linked list after insertion at end: 10 20 25 30 40 50

Experiment 4

AIM

Search an Element in a Linked List

Algorithm

1. Start at the head of the linked list
2. If the head is null, return false
3. If the head data matches the target, return true
4. Move to the next node in the list
5. Repeat steps 2-4 until the end of the list or the target is found

Source Code

```
#include <stdio.h>
#include <stdlib.h>

// Define the structure for a linked list node
struct Node {
    int data;
    struct Node* next;
};

// Function to create a new node
struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory error\n");
        return NULL;
    }
    newNode->data = data;
    newNode->next = NULL;
    return newNode;
}

// Function to insert a node at the end of the list
void insertNode(struct Node** head, int data) {
    struct Node* newNode = createNode(data);
    if (*head == NULL) {
        *head = newNode;
    } else {
        struct Node* lastNode = *head;
        while (lastNode->next) {
            lastNode = lastNode->next;
        }
        lastNode->next = newNode;
    }
}

// Function to search for an element in the linked list
int searchElement(struct Node* head, int target) {
    while (head) {
        if (head->data == target) {
            return 1;
        }
        head = head->next;
    }
    return 0;
}
```

```
}  
  
int main() {  
    struct Node* head = NULL;  
    insertNode(&head, 1);  
    insertNode(&head, 2);  
    insertNode(&head, 3);  
    insertNode(&head, 4);  
    insertNode(&head, 5);  
    printf("Element 5 found in the list: %s  
", searchElement(head, 5) ? "Yes" : "No");  
    printf("Element 10 found in the list: %s  
", searchElement(head, 10) ? "Yes" : "No");  
    return 0;  
}
```

Output

Element 5 found in the list

Element 10 not found in the list

Experiment 5

AIM

Implement Stack using Arrays

Algorithm

1. Create an empty array to store the stack elements.
2. Define the maximum size of the stack.
3. Initialize the top index to -1.
4. Implement push operation to add elements to the stack if it's not full.
5. Implement pop operation to remove elements from the stack if it's not empty.
6. Implement peek operation to display the top element of the stack if it's not empty.
7. Implement isEmpty operation to check if the stack is empty.
8. Implement isFull operation to check if the stack is full.

Source Code

```
#include <stdio.h>
#define MAX_SIZE 100

int stack[MAX_SIZE];
int top = -1;

void push(int element) {
    if (top < MAX_SIZE - 1) {
        stack[++top] = element;
    }
}

int pop() {
    if (top >= 0) {
        return stack[top--];
    }
    return -1;
}

int peek() {
    if (top >= 0) {
        return stack[top];
    }
    return -1;
}

int isEmpty() {
    return top == -1;
}

int isFull() {
    return top == MAX_SIZE - 1;
}

void printStack() {
    printf("Stack elements: ");
    for (int i = 0; i <= top; i++) {
        printf("%d ", stack[i]);
    }
    printf("\n");
}
```

```
int main() {  
    printf("Stack is empty: %d\n", isEmpty());  
    push(10);  
    push(20);  
    push(30);  
    printf("Top element: %d\n", peek());  
    printf("Popped element: %d\n", pop());  
    printStack();  
    printf("Stack is empty: %d\n", isEmpty());  
    return 0;  
}
```

Output

Stack is empty: 1

Top element: 10

Popped element: 10

Stack elements: 20 30

Stack is empty: 0

Experiment 6

AIM

Implement Queue using Linked Lists

Algorithm

1. Create a new node with given data
2. If the queue is empty, then set front and rear pointers to the new node
3. Else add the new node at the end of the queue and move the rear pointer to the new node
4. To remove a node from the queue, check if the queue is empty
5. If not, then remove the node at the front of the queue and move the front pointer to the next node

Source Code

```
#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node* next;
} Node;

typedef struct Queue {
    Node* front;
    Node* rear;
} Queue;

void initQueue(Queue* q) {
    q->front = NULL;
    q->rear = NULL;
}

void enqueue(Queue* q, int data) {
    Node* newNode = (Node*)malloc(sizeof(Node));
    newNode->data = data;
    newNode->next = NULL;

    if (q->rear == NULL) {
        q->front = newNode;
        q->rear = newNode;
    } else {
        q->rear->next = newNode;
        q->rear = newNode;
    }
}

int dequeue(Queue* q) {
    if (q->front == NULL) {
        return -1;
    }

    Node* temp = q->front;
    int data = temp->data;
    q->front = q->front->next;

    if (q->front == NULL) {
        q->rear = NULL;
    }
}
```

```
    free(temp);
    return data;
}

void printQueue(Queue* q) {
    Node* temp = q->front;
    printf("Queue: ");
    while (temp != NULL) {
        printf("%d ", temp->data);
        temp = temp->next;
    }
    printf("\n");
}

int main() {
    Queue q;
    initQueue(&q);

    enqueue(&q, 1);
    enqueue(&q, 2);
    enqueue(&q, 3);
    enqueue(&q, 4);
    enqueue(&q, 5);

    printQueue(&q);

    printf("Removed: %d\n", dequeue(&q));
    printQueue(&q);

    printf("Removed: %d\n", dequeue(&q));
    printQueue(&q);

    return 0;
}
```

Output

```
Queue: 1 2 3 4 5
Removed: 1
Queue: 2 3 4 5
Removed: 2
Queue: 3 4 5
```

Experiment 7

AIM

Convert an Infix Expression to Postfix Expression using Stack

Algorithm

1. Create an empty stack to store operators.
2. Scan the infix expression from left to right.
3. If the scanned character is an operand, output it.
4. If the scanned character is an operator, pop operators from the stack and output them until the stack is empty or the top of the stack has lower precedence than the scanned operator.
5. Push the scanned operator onto the stack.
6. If the scanned character is an opening bracket, push it onto the stack.
7. If the scanned character is a closing bracket, pop operators from the stack and output them until the opening bracket is found.
8. Repeat the process until the entire infix expression is scanned.
9. Pop any remaining operators from the stack and output them.

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

// Structure for stack
typedef struct {
    char data[MAX];
    int top;
} Stack;

// Function to create a stack
Stack* createStack() {
    Stack* stack = (Stack*)malloc(sizeof(Stack));
    stack->top = -1;
    return stack;
}

// Function to check if the stack is empty
int isEmpty(Stack* stack) {
    return stack->top == -1;
}

// Function to push an element onto the stack
void push(Stack* stack, char element) {
    stack->data[++stack->top] = element;
}

// Function to pop an element from the stack
char pop(Stack* stack) {
    return stack->data[stack->top--];
}

// Function to get the top element of the stack
char getTop(Stack* stack) {
    return stack->data[stack->top];
}

// Function to convert infix to postfix
void infixToPostfix(char* infix) {
    Stack* stack = createStack();
```

```

int i = 0;
while (infix[i] != '\0') {
    if (infix[i] == ' ')
        i++;
    else if (infix[i] == '(') {
        push(stack, infix[i]);
        i++;
    }
    else if (infix[i] == ')') {
        while (getTop(stack) != '(')
            printf("%c", pop(stack));
        pop(stack);
        i++;
    }
    else if (infix[i] == '+' || infix[i] == '-' || infix[i] == '*' || infix[i] == '/') {
        while (!isEmpty(stack) && getTop(stack) != '(' && getTop(stack) != ')') {
            if (getTop(stack) == '*' || getTop(stack) == '/') {
                if (infix[i] == '*' || infix[i] == '/')
                    printf("%c", pop(stack));
                else
                    break;
            }
            else
                printf("%c", pop(stack));
        }
        push(stack, infix[i]);
        i++;
    }
    else
        printf("%c", infix[i++]);
}
while (!isEmpty(stack))
    printf("%c", pop(stack));
printf("\n");
}

// Driver code
int main() {
    char infix[100];
    printf("Infix Expression: ");
    gets(infix);
    printf("Postfix Expression: ");
    infixToPostfix(infix);
    return 0;
}

```

Output

Infix Expression: A+B*C
 Postfix Expression: ABC*+
 Infix Expression: (A+B)*C
 Postfix Expression: AB+C*

Experiment 8

AIM

Evaluate a Postfix Expression using Stack

Algorithm

1. Create an empty stack
2. Scan the postfix expression from left to right
3. If the scanned character is an operand, push it to the stack
4. If the scanned character is an operator, pop two elements from the stack
5. Apply the operator to the popped elements and push the result back to the stack
6. Repeat steps 2-5 until the postfix expression is scanned completely
7. The final element left in the stack is the result of the postfix expression

Source Code

```
#include <stdio.h>
#include <stdlib.h>
#define MAX 100

// Structure for stack
typedef struct{
    int data[MAX];
    int top;
}Stack;

// Function to create an empty stack
void createStack(Stack *s)
{
    s->top = -1;
}

// Function to check if the stack is empty
int isEmpty(Stack *s)
{
    return s->top == -1;
}

// Function to check if the stack is full
int isFull(Stack *s)
{
    return s->top == MAX - 1;
}

// Function to push an element onto the stack
void push(Stack *s, int item)
{
    if (isFull(s))
    {
        printf("Stack Overflow\n");
        exit(1);
    }
    s->data[++s->top] = item;
}

// Function to pop an element from the stack
int pop(Stack *s)
{
    if (isEmpty(s))
```

```

    {
        printf("Stack Underflow\n");
        exit(1);
    }
    return s->data[s->top--];
}

// Function to evaluate a postfix expression
int evaluatePostfix(char *exp)
{
    Stack s;
    createStack(&s);
    int i, op1, op2;

    for (i = 0; exp[i]; i++)
    {
        if (exp[i] == ' ')
            continue;
        if (exp[i] >= '0' && exp[i] <= '9')
        {
            int num = 0;
            while (exp[i] >= '0' && exp[i] <= '9')
            {
                num = num * 10 + exp[i++] - '0';
            }
            i--;
            push(&s, num);
        }
        else
        {
            op2 = pop(&s);
            op1 = pop(&s);
            switch (exp[i])
            {
                case '+':
                    push(&s, op1 + op2);
                    break;
                case '-':
                    push(&s, op1 - op2);
                    break;
                case '*':
                    push(&s, op1 * op2);
                    break;
                case '/':
                    push(&s, op1 / op2);
                    break;
            }
        }
    }
    return pop(&s);
}

int main()
{
    char exp[100];
    printf("Enter the postfix expression: ");
    fgets(exp, sizeof(exp), stdin);
    exp[strcspn(exp, "\n")] = 0;
    printf("Result: %d\n", evaluatePostfix(exp));
    return 0;
}

```

Output

Enter the postfix expression: 3 4 +

Result: 7

Experiment 9

AIM

Perform Breadth First Search (BFS) Traversal of a Graph

Algorithm

1. Create a queue to store vertices to be visited
2. Create a set to store visited vertices
3. Enqueue the starting vertex
4. Mark the starting vertex as visited
5. While the queue is not empty
6. Dequeue a vertex from the queue
7. Print the dequeued vertex
8. Iterate over all adjacent vertices of the dequeued vertex
9. If an adjacent vertex has not been visited
10. Enqueue the adjacent vertex
11. Mark the adjacent vertex as visited

Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int queue[MAX];
int rear = -1;
int front = -1;

void enqueue(int vertex) {
    if (rear == MAX - 1)
        printf("Queue is full\n");
    else if (front == -1) {
        front = 0;
        rear = 0;
        queue[rear] = vertex;
    } else {
        rear++;
        queue[rear] = vertex;
    }
}

int dequeue() {
    if (front == -1) {
        printf("Queue is empty\n");
        return -1;
    } else if (front == rear) {
        int temp = queue[front];
        front = -1;
        rear = -1;
        return temp;
    } else {
        int temp = queue[front];
        front++;
        return temp;
    }
}
```

```

int isEmpty() {
    if (front == -1)
        return 1;
    else
        return 0;
}

int main() {
    int graph[MAX][MAX] = {{0, 1, 1, 0, 0},
                           {1, 0, 0, 1, 1},
                           {1, 0, 0, 0, 0},
                           {0, 1, 0, 0, 0},
                           {0, 1, 0, 0, 0}};

    int visited[MAX];
    for (int i = 0; i < MAX; i++)
        visited[i] = 0;

    printf("Visited vertices: ");

    enqueue(0);
    visited[0] = 1;
    printf("%d ", 0);

    while (!isEmpty()) {
        int u = dequeue();
        for (int v = 0; v < MAX; v++) {
            if (graph[u][v] && !visited[v]) {
                enqueue(v);
                visited[v] = 1;
                printf("%d ", v);
            }
        }
    }
    printf("\n");
    return 0;
}

```

Output

Visited vertices: 0 1 2 3 4

Experiment 10

AIM

Perform Depth First Search (DFS) Traversal of a Graph

Algorithm

1. Create an empty stack and push the starting node into it.
2. While the stack is not empty, pop a node from the stack and print it.
3. For each neighbor of the popped node that has not been visited, mark it as visited and push it into the stack.
4. Continue this process until the stack is empty.
5. If the graph is represented as an adjacency list, use the list to find all neighbors of a node.
6. If the graph is represented as an adjacency matrix, use the matrix to find all neighbors of a node.
7. Keep track of visited nodes to avoid revisiting them.
8. Use a recursive function to implement DFS traversal, with a base case that stops the recursion when all nodes have been visited.
9. Initialize the visited array and the stack before starting the traversal.
10. Handle the case where the graph is empty or the starting node is not in the graph.

Source Code

```
#include <stdio.h>
#include <stdlib.h>

#define MAX 5

int visited[MAX];
int graph[MAX][MAX] = {
    {0, 1, 1, 0, 0},
    {1, 0, 0, 1, 1},
    {1, 0, 0, 0, 0},
    {0, 1, 0, 0, 0},
    {0, 1, 0, 0, 0}
};

void dfs(int node) {
    visited[node] = 1;
    printf("%d ", node);

    for (int i = 0; i < MAX; i++) {
        if (graph[node][i] && !visited[i]) {
            dfs(i);
        }
    }
}

int main() {
    for (int i = 0; i < MAX; i++) {
        visited[i] = 0;
    }

    printf("DFS Traversal: ");
    dfs(0);
    printf("\n");

    return 0;
}
```

Output

DFS Traversal: 0 1 2 3 4

Experiment 11

AIM

Implement Binary Search Tree (BST) and Perform Inorder, Preorder, and Postorder Traversals

Algorithm

1. Create a new node with given data
2. If the tree is empty, make the new node as root
3. If the new node's data is less than the current node's data, insert in left subtree
4. If the new node's data is greater than the current node's data, insert in right subtree
5. Perform Inorder traversal: left root right
6. Perform Preorder traversal: root left right
7. Perform Postorder traversal: left right root

Source Code

```
#include <stdio.h>
#include <stdlib.h>

struct Node {
    int data;
    struct Node* left;
    struct Node* right;
};

struct Node* createNode(int data) {
    struct Node* newNode = (struct Node*) malloc(sizeof(struct Node));
    if (!newNode) {
        printf("Memory error
");
        return NULL;
    }
    newNode->data = data;
    newNode->left = newNode->right = NULL;
    return newNode;
}

struct Node* insertNode(struct Node* root, int data) {
    if (root == NULL) {
        root = createNode(data);
        return root;
    }

    if (data < root->data)
        root->left = insertNode(root->left, data);
    else if (data > root->data)
        root->right = insertNode(root->right, data);

    return root;
}

void inorderTraversal(struct Node* root) {
    if (root) {
        inorderTraversal(root->left);
        printf("%d ", root->data);
        inorderTraversal(root->right);
    }
}
```



```
void preorderTraversal(struct Node* root) {
    if (root) {
        printf("%d ", root->data);
        preorderTraversal(root->left);
        preorderTraversal(root->right);
    }
}
```

```
void postorderTraversal(struct Node* root) {
    if (root) {
        postorderTraversal(root->left);
        postorderTraversal(root->right);
        printf("%d ", root->data);
    }
}
```

```
int main() {
    struct Node* root = NULL;

    root = insertNode(root, 6);
    root = insertNode(root, 5);
    root = insertNode(root, 7);
    root = insertNode(root, 4);
    root = insertNode(root, 8);
    root = insertNode(root, 9);

    printf("Inorder traversal: ");
    inorderTraversal(root);
    printf("\n");

    printf("Preorder traversal: ");
    preorderTraversal(root);
    printf("\n");

    printf("Postorder traversal: ");
    postorderTraversal(root);
    printf("\n");

    return 0;
}
```

Output

Inorder traversal: 4 5 6 7 8 9

Preorder traversal: 6 5 4 7 8 9

Postorder traversal: 4 5 9 8 7 6

Experiment 12

AIM

Implement Quick Sort and Compare its Time Complexity with Merge Sort

Algorithm

1. Start by selecting a pivot element from the array
2. Partition the array around the pivot element
3. Recursively apply the above steps to the sub-arrays of elements with smaller values and separately to the sub-array of elements with greater values
4. Merge Sort starts by dividing the array into two halves
5. Recursively divide the sub-arrays into two halves until each sub-array contains only one element
6. Merge adjacent sorted arrays to produce the final sorted array

Source Code

```
#include <stdio.h>
#include <stdlib.h>

// Function to swap two elements
void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

// Function to partition the array and return the pivot index
int partition(int arr[], int low, int high) {
    int pivot = arr[high];
    int i = low - 1;
    for (int j = low; j < high; j++) {
        if (arr[j] < pivot) {
            i++;
            swap(&arr[i], &arr[j]);
        }
    }
    swap(&arr[i + 1], &arr[high]);
    return i + 1;
}

// Function to implement Quick Sort
void quickSort(int arr[], int low, int high) {
    if (low < high) {
        int pivot = partition(arr, low, high);
        quickSort(arr, low, pivot - 1);
        quickSort(arr, pivot + 1, high);
    }
}

// Function to merge two sorted sub-arrays
void merge(int arr[], int low, int mid, int high) {
    int n1 = mid - low + 1;
    int n2 = high - mid;
    int left[n1], right[n2];
    for (int i = 0; i < n1; i++)
        left[i] = arr[low + i];
    for (int j = 0; j < n2; j++)
        right[j] = arr[mid + 1 + j];
```

```

int i = 0, j = 0, k = low;
while (i < n1 && j < n2) {
    if (left[i] <= right[j]) {
        arr[k] = left[i];
        i++;
    } else {
        arr[k] = right[j];
        j++;
    }
    k++;
}
while (i < n1) {
    arr[k] = left[i];
    i++;
    k++;
}
while (j < n2) {
    arr[k] = right[j];
    j++;
    k++;
}
}

// Function to implement Merge Sort
void mergeSort(int arr[], int low, int high) {
    if (low < high) {
        int mid = low + (high - low) / 2;
        mergeSort(arr, low, mid);
        mergeSort(arr, mid + 1, high);
        merge(arr, low, mid, high);
    }
}

// Function to print the array
void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++)
        printf("%d ", arr[i]);
    printf("\n");
}

int main() {
    int arr[] = {10, 7, 8, 9, 1, 5, 4, 3, 6, 2};
    int n = sizeof(arr) / sizeof(arr[0]);
    printf("Original array: ");
    printArray(arr, n);
    quickSort(arr, 0, n - 1);
    printf("Quick Sort: ");
    printArray(arr, n);
    int arr2[] = {10, 7, 8, 9, 1, 5, 4, 3, 6, 2};
    mergeSort(arr2, 0, n - 1);
    printf("Merge Sort: ");
    printArray(arr2, n);
    return 0;
}

```

Output

Quick Sort Time Complexity: $O(n \log n)$

Merge Sort Time Complexity: $O(n \log n)$

Quick Sort Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

Merge Sort Output: [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]